Suppose, though, that you don't have the data but a function that returns the data:

```
getPhoneNumbers = -> phoneNumbers
```

You'll remember writing fluent interfaces in chapter 7. Here goes another one; suppose you call it `withResultOf`:

```
withResultOf(getPhoneNumbers)
.filter(relationshipIs 'friend')
```

The `withResultOf` function will need to return an object that has a `filter` method:

```
withResultOf = (fn) ->
  filter: ->
```

At some point in the program, you'll need the actual data itself. Call this method that returns the data `evaluate`:

```
withResultOf = (fn) ->
  runInOrder = []
  {
    filter: (filterFn) ->
      runInOrder.push (data) -> item for item in data when filterFn item
      @
    evaluate: ->
      data = fn()
      for processFn in runInOrder
        processFn data
  }
```

Returning "this" to allow chaining for the fluent interface

By now, this is a familiar pattern: some imperative code hidden behind a largely functional interface:

```
withResultOf(getPhoneNumbers)
.filter(relationshipIs 'friend')
.evaluate()

# [
#   { name: 'hannibal', number: '555-5551', relationship: 'friend'
},
#   { name: 'hal_9000', number: 'disconnected', relationship:
'friend' },
```